

NAME: FAATINA S
DATE:25.01.2026

REG NO:3122 23 5001 046
EX NO: 4

ASSIGNMENT -4

Memory Match Game using DOM, HTML and JavaScript

AIM

To design and implement a Memory Match Game using HTML, CSS, and JavaScript, where the player matches pairs of cards within a given time limit by applying DOM manipulation, event handling, and basic game logic.

CODE

index.html

```
<!DOCTYPE html>
<html>
  <HEAD>
    <meta charset="UTF-8">
    <title>Memory Match Game</title>
    <link rel="stylesheet" href="style.css">
  </HEAD>
  <body>
    <h1 style="color: #db68c4;background-color: white;">MEMORY MATCH GAME</h1>
    <div class="controls">
      <label for="level" style="color: rgb(6, 143, 70);font-size:130%;font-weight: 1000;">Select level:</label>
      <select id="level">
        <option value="easy">Easy: 2 pairs, 15 sec</option>
        <option value="medium">Medium: 4 pairs, 30 sec</option>
        <option value="hard">Hard: 6 pairs, 45 sec</option>
      </select>
      <button onclick="startGame()" style="color: rgb(6, 143, 70);font-size:90%;font-weight: 500;">Start</button>
      <button onclick="restartGame()" style="color: rgb(6, 143, 70);font-size:90%;font-weight: 500;">Restart</button>
    </div>
    <p class="timer" style="color:#930075;font-size:120%;font-weight: 1000;">
      Time left:<span id="time">0</span>sec
    </p>
    <div id="game-board"></div>
    <script src="script.js"></script>
  </body>
</html>
```

Script.js

```
let firstCard=null;
let secondCard=null;
let lock=false;
let matchedPairs = 0;
let total_pairs=0;
let time_left=0;
let timer;

const board=document.getElementById("game-board");
const timeDisplay=document.getElementById("time");

const symbols = ["🍪", "🕒", "💎", "🕒", "🍉", "🍔"];

function startGame(){
    clearInterval(timer);
    resetGame();
    const level=document.getElementById("level").value;
    if (level==="easy"){
        total_pairs=2;
        time_left=15;
    }
    else if (level === "medium")
    {
        total_pairs = 4;
        time_left = 30;
    }
    else
    {
        total_pairs = 6;
        time_left = 45;
    }
    timeDisplay.textContent=time_left;
    createBoard(total_pairs);
    startTimer();
}

function restartGame(){
    clearInterval(timer);
    startGame();
}

function createBoard(pairs){
    board.innerHTML="";
    board.style.gridTemplateColumns=`repeat(${pairs * 2 <= 4 ? 2 : 4}, 100px)`;
    let selected=symbols.slice(0,pairs);
    let cardValues=[...selected,...selected];
    shuffle(cardValues);
}
```

```

    cardValues.forEach(symbol =>{
        const card=document.createElement("div");
        card.classList.add("card");
        card.dataset.symbol=symbol;
        card.textContent="?";
        card.addEventListener("click",()=>flipCard(card));
        board.appendChild(card);
    });
}
function shuffle(array){
    for(let i=array.length-1;i>0;i--){
        let j=Math.floor(Math.random()*(i+1));
        [array[i],array[j]]=array[j],array[i];
    }
}
function flipCard(card){
    if(lock || card===firstCard || card.classList.contains("matched"))
        return;
    card.classList.add("flip");
    card.textContent=card.dataset.symbol;
    if(!firstCard)
        firstCard=card;
    else{
        secondCard=card;
        checkMatch();
    }
}
function checkMatch() {
    const isMatch=firstCard.dataset.symbol=== secondCard.dataset.symbol;
    if(isMatch){
        firstCard.classList.add("matched");
        secondCard.classList.add("matched");
        matchedPairs++;
        resetSelection();
        if(matchedPairs===total_pairs){
            clearInterval(timer);
            lock=true;
            setTimeout(()=> alert("WON, All pairs matched"),300);
        }
    }
    else{
        lock=true;
        setTimeout(() => {
            firstCard.classList.remove("flip");
            secondCard.classList.remove("flip");
            firstCard.textContent="?";
            secondCard.textContent="?";
            resetSelection();

```

```

        }, 500);
    }
}

function resetSelection() {
    firstCard=null;
    secondCard=null;
    lock=false;
}

function startTimer(){
    timer=setInterval(() => {
        time_left--;
        timeDisplay.textContent=time_left;
        if(time_left===0){
            clearInterval(timer);
            lock=true;
            alert("LOST, Time is up");
        }
    }, 1000);
}

function resetGame(){
    board.innerHTML="";
    firstCard=null;
    secondCard=null;
    lock=false;
    matchedPairs=0;
}

```

Style.css

```

body{
    background-color: #f7c4ed;
    text-align: center;
    font-family:Arial, sans-serif;
}

.controls{
    margin-top: 60px;
}

#game-board{
    display: grid;
    gap: 15px;
    justify-content: center;
    margin-top: 60px;
}

.card{

```

```

width: 100px;
height: 150px;
background-color: #1e90ff;
color: white;
display: flex;
font-size: 70px;
justify-content:center ;
align-items: center;
border-radius: 8%;
cursor:pointer;
user-select: none;
}

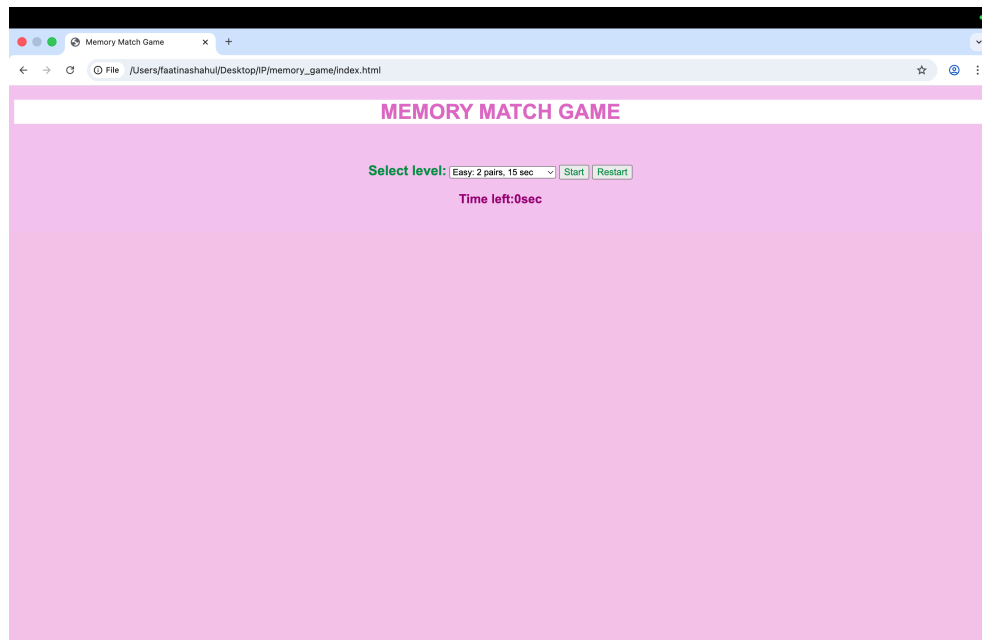
.card.flip {
background-color: white;
color: black;
cursor: default;
}

.card.matched {
background-color: #2ecc71;
color: white;
cursor: default;
}

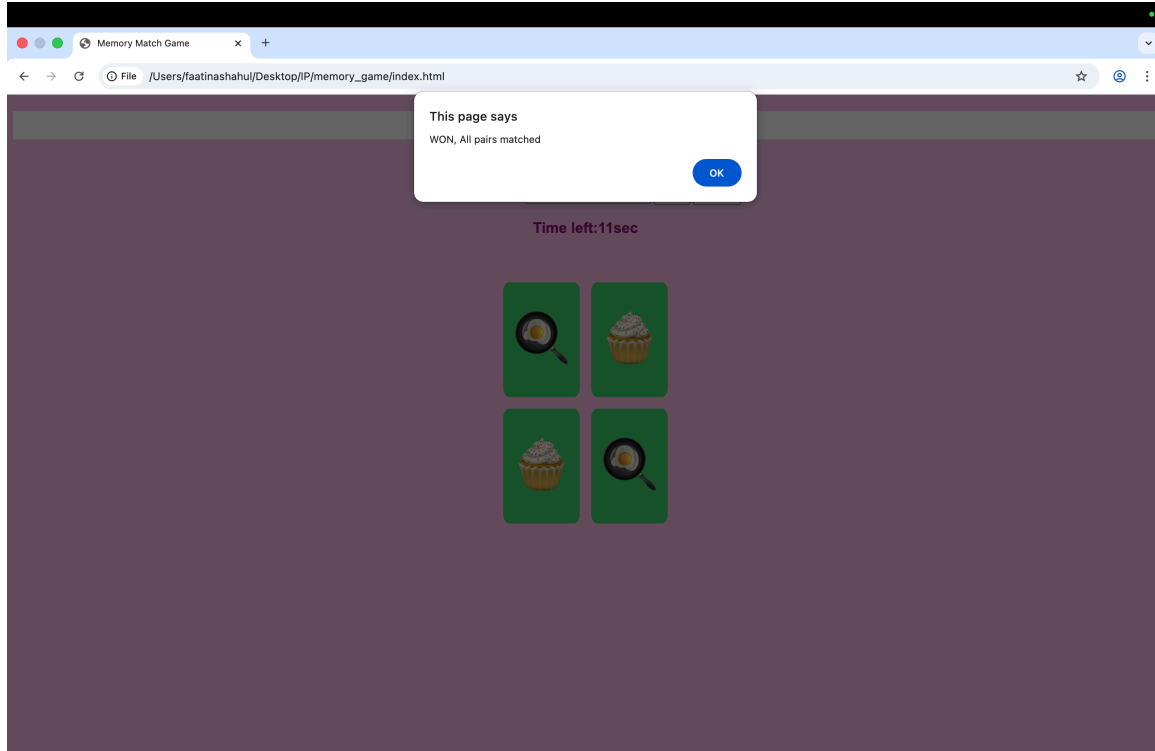
```

OUTPUT

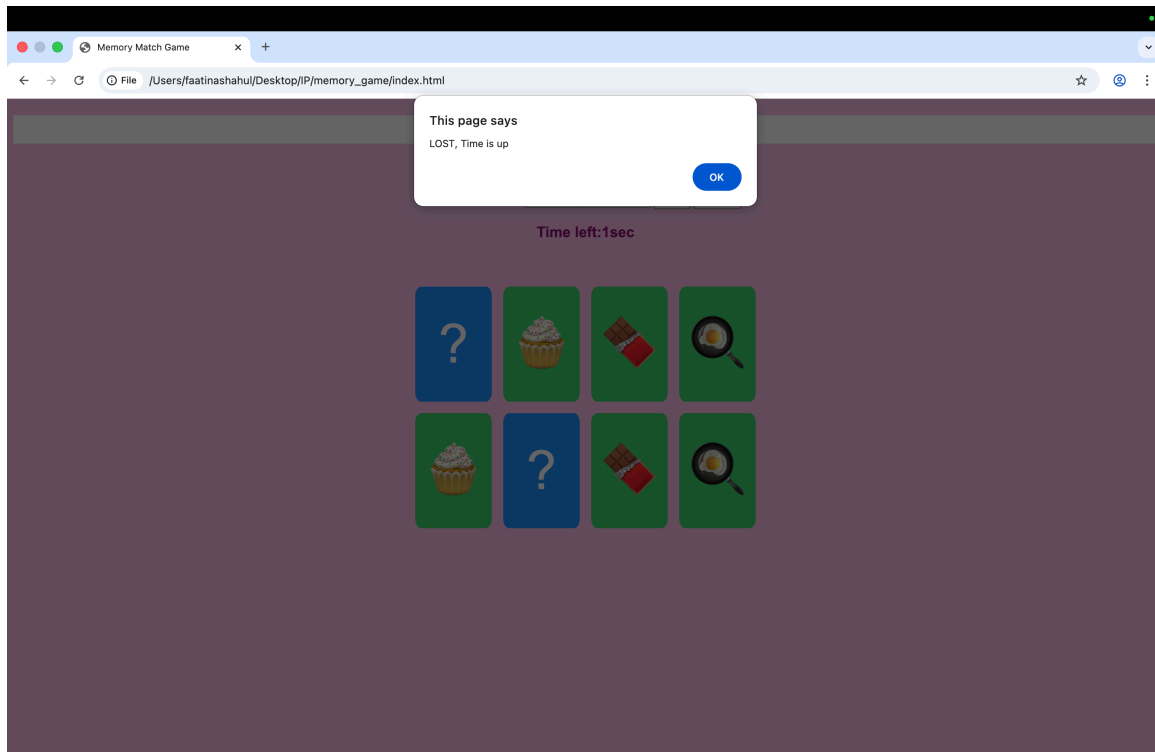
Initial Page



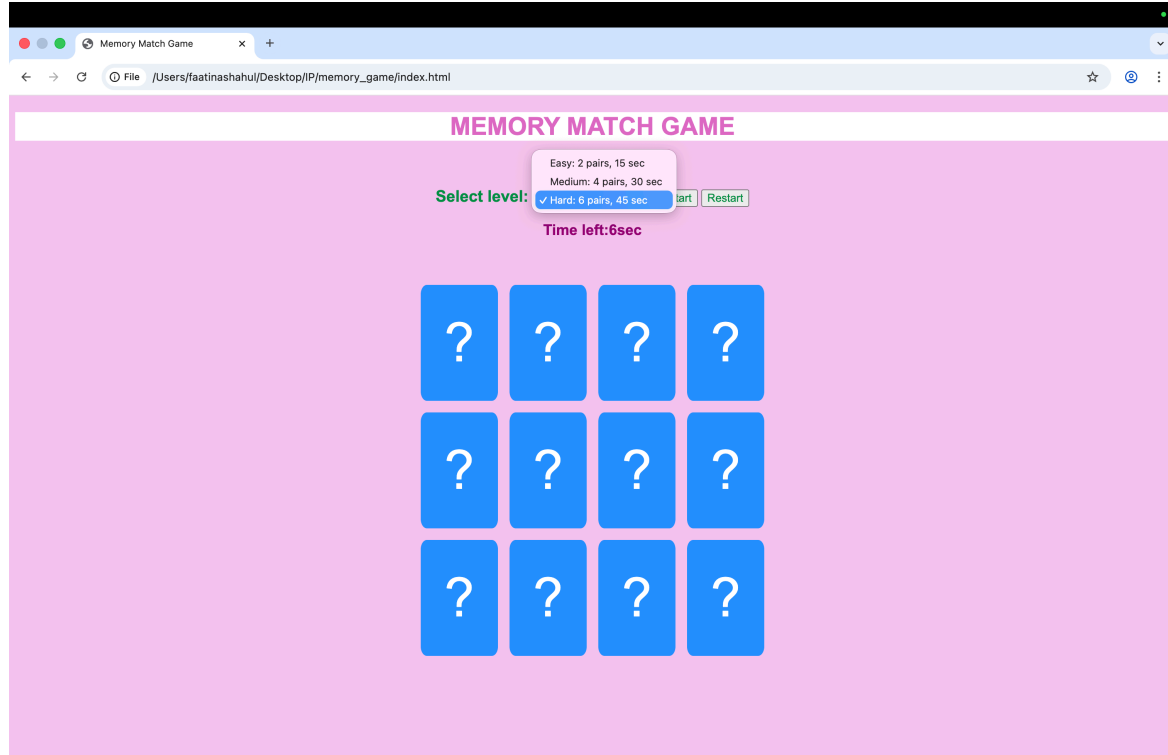
EASY LEVEL CODE PAGE



MEDIUM LEVEL CODE PAGE



DIFFICULT LEVEL CODE PAGE



RESULT

The Memory Match Game was successfully developed and executed. The application correctly displays the card layout, allows card flipping and matching, manages time-based gameplay, and accurately determines win or loss conditions. The restart functionality works as expected, and the game performs according to the specified difficulty levels.

LEARNING OUTCOME

1. Understood and apply HTML **DOM** manipulation for dynamic content creation.
2. Implemented **event handling** using JavaScript for user interactions.
3. Used **conditional logic and control structures** to manage game flow.
4. Applied **timers** (setInterval, clearInterval) in real-time applications.
5. Designed responsive layouts using **CSS Grid**.
6. Developed problem-solving skills by implementing game logic and state management.